Communication Lower Bounds for Multi-Party Graph Traversal

Quarter 2 Project Report

Ryan Batubara rbatubara@ucsd.edu
Ivy Hawks mahawks@ucsd.edu
Darren Ho dah103@ucsd.edu
Ciro Zhang ciz001@ucsd.edu

Mentor: Dr. Shachar Lovett
slovett@ucsd.edu

Abstract

TBD

Repository: https://github.com/rybplayer/DSCCapstone

1 Introduction

This report studies communication lower and upper bounds for multi-party graph traversal problems, motivated by real systems in which no single agent holds the full network state. We focus on settings where a global graph is known but each party privately owns a subset of vertices and edges, and the task is to determine reachability or shortest paths that must lie within the union of their private subgraphs.

1.1 Motivation from multi-party traversal case studies

Large-scale transportation and infrastructure disruptions create a natural model for multi-party graph traversal. Consider the 2010 European volcanic ash shutdown: air-navigation service providers, airports, and airlines each control disjoint portions of airspace and schedules. Rerouting passengers is a shortest-path problem in a layered graph whose feasible vertices and edges change over time. No party can broadcast its full real-time state, both for operational and privacy reasons, so route feasibility must be computed with limited information exchange.

Similar graph-traversal constraints appear in urban transit shutdowns and phased restorations, where distinct agencies own stations and tunnels. After Hurricane Sandy, service availability evolved over weeks, and agencies coordinated alternate routes under partial knowledge of what infrastructure was open. Highway detours after the I-95 bridge collapse in Philadelphia illustrate the same phenomenon in a road network: different operators own highway segments and toll roads, and a single failed cut edge forces traffic onto alternative paths with asymmetric knowledge of capacities.

The maritime and logistics examples are equally instructive. The 2021 Suez Canal blockage forced carriers to reroute around the Cape of Good Hope or wait, a decision that depends on schedule and capacity information held privately by shipping lines and port operators. When the Baltimore Key Bridge collapsed, access to the port was blocked and cargo was diverted to alternate ports, again requiring coordination across a sea–port–land graph with private operational constraints.

Communication bottlenecks become even sharper in critical infrastructure networks. During the 2021 Texas ERCOT blackout, grid operators and utilities effectively solve a constrained flow and connectivity problem as generation and transmission nodes go offline. The Colonial Pipeline cyber disruption similarly forced fuel logistics to reroute through alternate depots and terminals with incomplete visibility. Finally, the 2021 Meta backbone withdrawal and the 2016 Dyn DNS attack show that digital services rely on reachability in routing and dependency graphs, where policy and security constraints limit information sharing across administrative domains.

These cases motivate a theoretical study of multi-party graph traversal: agents must collaboratively decide reachability and shortest paths while their private subgraphs, capacities, or weights are not fully shareable. The fundamental question is how much communication is necessary, and how randomized protocols, sketches, or partial disclosures can reduce the

burden while preserving correctness.

1.2 Why communication is the bottleneck

In the model we consider, each party has unlimited local computation but limited ability to share data. This is realistic: air traffic control centers, network operators, or utilities can compute detailed local routes, but cannot afford to broadcast all local states and constraints due to bandwidth, privacy, regulatory, or security limitations. The communication cost, not the local computation, is what constrains real-time coordination.

Communication complexity makes this separation explicit. In particular, it shows that even when each party can compute arbitrarily powerful local summaries, some global questions still require a large number of bits to be exchanged. For example, deciding whether two privately held sets intersect already requires linear communication in the worst case. This lower bound appears again in many traversal problems via reductions that embed set disjointness or equality into graph reachability. Thus, communication is not simply a practical nuisance: it is an inherent barrier that must be designed around.

1.3 Why bounds matter for hardware, networks, and algorithms

Lower and upper bounds provide practical guidance in three ways. First, they inform hardware design. If a reachability or shortest-path query provably requires $\Omega(n)$ bits of communication, then any system intended to answer such queries at scale must provision sufficient bandwidth or memory to move that information. Second, bounds guide network architecture: if communication is unavoidable, the topology of communication links, the placement of caches, and the choice of aggregation points become part of the algorithmic design. Third, bounds shape algorithm development: a tight lower bound rules out overly ambitious protocols and motivates either approximate solutions, randomized protocols with small error, or problem relaxations that admit sublinear communication. Conversely, upper bounds show which tasks are feasible under realistic constraints, making them valuable to practitioners who must trade correctness, latency, and privacy.

1.4 Communication complexity basics

We briefly formalize the model. Let $f : \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ be a function. In the two-party model, Alice receives $x \in \mathcal{X}$ and Bob receives $y \in \mathcal{Y}$. A deterministic protocol is a binary decision tree whose nodes are labeled by which party speaks and whose edges are labeled by bits; the transcript is the sequence of bits exchanged. The protocol must output $f(x,y)$ at a leaf. The deterministic communication complexity $D(f)$ is the minimum, over all correct protocols, of the maximum transcript length over all inputs.

Randomized protocols allow parties to use random coins to reduce communication. A public-coin protocol assumes a shared random string, while a private-coin protocol lets

each party sample independently. Fixing the randomness yields a deterministic protocol, so a randomized protocol is a distribution over deterministic protocols. The randomized communication complexity $R_\varepsilon(f)$ is the minimum number of communicated bits needed to compute f with error at most ε on every input. Public-coin and private-coin models are equivalent up to small overhead, and randomness often trades a small probability of error for large savings in communication.

Two classical benchmark problems are equality and disjointness. Equality asks whether $x = y$ for $x, y \in \{0,1\}^n$, and disjointness asks whether $X \cap Y = \emptyset$ for subsets of $[n]$. Both are complete for a wide range of lower-bound techniques. Deterministically, equality requires n bits in the worst case, and disjointness requires $\Omega(n)$ communication. Randomization yields large gains for equality via hashing, but not for disjointness, which remains hard.

1.5 From communication to graph traversal

Graph traversal problems can be embedded with these benchmarks. A simple variant illustrates the connection. Let a known undirected graph $G = (V, E)$ be given. Alice receives a start vertex v and a subset $X \subseteq V$, Bob receives an end vertex w and a subset $Y \subseteq V$. The task is to output 1 if there exists a path from v to w using only vertices in $X \cup Y$ and edges that are wholly contained within X or wholly contained within Y. By constructing a graph in which each bit of an input string controls which local vertices are present, one can reduce equality to this traversal problem. This implies that even this restricted reachability task has deterministic communication complexity $\Omega(n)$ when $|V| = \Theta(n)$.

The case studies above map naturally to this model. Each actor controls a subgraph of the global infrastructure; edges represent feasible transitions, and reachability encodes whether a route exists that respects ownership and availability constraints. The reduction perspective explains why coordination can be intrinsically expensive: some instances encode hard communication problems, so no protocol can be both exact and frugal with communication in the worst case.

1.6 Scope of this report

The remainder of this report develops a communication-complexity view of multi-party graph traversal. We formalize traversal models, establish lower bounds via reductions, and explore upper bounds that exploit randomness, sketches, or structure. Throughout, the guiding message is that communication is not merely an implementation detail. It is a fundamental resource with provable limits, and understanding those limits yields actionable insights for designing distributed routing, resilient infrastructure, and scalable coordination protocols.

2 Reasearch

In order to better understand the complexity of our problem, we examined several related areas of theoretical computer science and algorithms. In particular, we focused on reductions from known hard problems, communication complexity, and lower bounds in distributed or constrained graph settings. Studying these connections helps determine whether efficient algorithms are likely to exist and provides a framework for proving hardness results.

The goal of this section is to summarize relevant ideas from existing literature and describe how they inform our approach. The main themes we explore are:

- Reductions from 3SAT to graph problems,
- Randomized communication protocols,
- Lower bounds for shortest path computation in generalized models.

Each of these areas provides insight into different aspects of our problem, including computational hardness, communication requirements, and structural constraints.

2.1 3SAT to Graph Reduction

A standard method for proving that a graph problem is NP-hard is to reduce from **3SAT**. Because 3SAT is NP-complete and highly structured, it serves as a canonical starting point for hardness reductions.

Goal. Given a 3SAT formula φ, we construct a graph $G(\varphi)$ such that

$$\varphi \text{ is satisfiable } \iff G(\varphi) \text{ has a feasible solution.}$$

If such a construction can be performed in polynomial time, then the graph problem is at least as hard as 3SAT.

3SAT definition. A 3SAT instance consists of Boolean variables $x_1, \ldots, x_n$ and clauses

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}),$$

where each literal ℓ is either x_i or $\neg x_i$. The objective is to determine whether there exists a truth assignment that satisfies all clauses.

Reduction structure. Most reductions from 3SAT to graph problems follow a common template:

- **Variable gadgets:** Each variable is represented by a subgraph that can be in one of two modes, corresponding to True or False.

- **Clause gadgets:** Each clause is represented by a subgraph that is feasible only if at least one of its literals evaluates to True.
- **Consistency connections:** Edges connect variable gadgets to clause gadgets so that all occurrences of a variable respect the same truth assignment.

Correctness. A valid reduction must show two directions:

- **Completeness:** If the formula is satisfiable, we can configure each variable gadget according to the assignment, producing a feasible solution in the graph.
- **Soundness:** If the graph has a feasible solution, the chosen configuration of variable gadgets determines an assignment that satisfies all clauses.

Size requirements. To ensure the reduction is efficient, the graph must be polynomial in size:

$$|V|, |E| = O(n + m),$$

where n is the number of variables and m is the number of clauses.

Relevance. These techniques are useful for our setting because they provide a blueprint for encoding logical constraints into graph structure. If our problem can simulate variable choices and clause satisfaction, then a similar reduction could be used to prove NP-hardness.

2.2 Randomized Protocols and Communication

Another perspective on our problem comes from communication complexity. In many settings, information about a graph is distributed across multiple agents or data sources. In such cases, randomness can help reduce the amount of communication required to coordinate computations.

Randomized protocols. A randomized protocol can be viewed as a distribution over deterministic protocols. Instead of following a fixed sequence of messages, parties use random bits to guide their decisions. This often allows problems to be solved with much less communication than deterministic approaches.

Public randomness. Public randomness refers to a shared random string known to all parties. This allows them to coordinate actions without explicitly communicating every detail. For example, in the equality problem, two parties can hash their inputs using a shared random function and compare only the hash values. This reduces the amount of data that must be transmitted while still achieving high accuracy.

Private randomness. In private randomness models, each party has its own random bits. While this is more realistic in practice, it is slightly weaker than public randomness because parties may need to communicate their random choices. Nevertheless, private randomness still reduces communication requirements compared to deterministic protocols.

Implications. Randomization can dramatically reduce communication complexity. For problems involving distributed graph information, randomized protocols may provide efficient coordination strategies even when deterministic communication would be too expensive.

2.3 Hardness of Shortest Path Variants

We also examined known lower bounds for shortest path problems under more general models. While shortest path computation is easy in centralized graphs, it becomes significantly harder when the graph is distributed, streamed, or represented using hyperedges.

Hypergraph setting. In directed hypergraphs, a single edge can connect multiple vertices simultaneously. Computing a minimum-weight hyperpath in this model is known to be NP-hard. This suggests that if our problem requires combining edges from multiple sources in a hypergraph-like structure, efficient algorithms may not exist.

Distributed setting. When graph data is distributed across multiple players and communication per round is limited, computing exact shortest paths requires many rounds of communication. Even though centralized algorithms are efficient, distributed protocols face inherent communication bottlenecks.

Streaming setting. If edges arrive as a stream and memory is limited, computing reachability or shortest paths requires either large memory or multiple passes over the data. These results show that strong lower bounds exist when information is spread across multiple sources.

Key takeaway. Shortest path problems become significantly more difficult when graph information is partitioned across agents, streams, or hyperedges. These results suggest that our problem may inherit similar lower bounds if it involves distributed or constrained graph information.

2.4 Summary

The literature reviewed here provides several important insights:

- Reductions from 3SAT offer a standard method for proving NP-hardness.
- Randomized protocols can reduce communication in distributed settings.
- Strong lower bounds exist for shortest path computation in generalized models such as hypergraphs, distributed systems, and streaming frameworks.

Together, these ideas help frame our problem within a broader theoretical context. They suggest both potential strategies for proving hardness and possible directions for designing algorithms under realistic constraints.

3 Findings

In this section we will explore our findings from our research.

3.1 Problem

Consider a graph G with edges E and vertices V, and subgraphs $G_1, \ldots, G_k \subset G$. Let $G_i = (E_i, V_i)$ for these subgraphs. For now let us suppose the graph is unweighted and undirected. The subgraphs need not cover G and can overlap. Each player i knows G and G_i, but not the other player's graphs. (This is akin to a map of a city, where each player is a different regional taxi company.) The goal is, given some starting vertex $v \in V$, and some end vertex, $w \in V$ to:

1. Identify if it is possible to construct a path $v, v_1, \ldots, v_l, w$ such that every edge on the path is in $\bigcup_{i=0}^{k} E_i$.
2. If such a path exists, find the minimum number of edges needed to get from v to w, while using only edges belonging to $\bigcup_{i=0}^{k} E_i$.

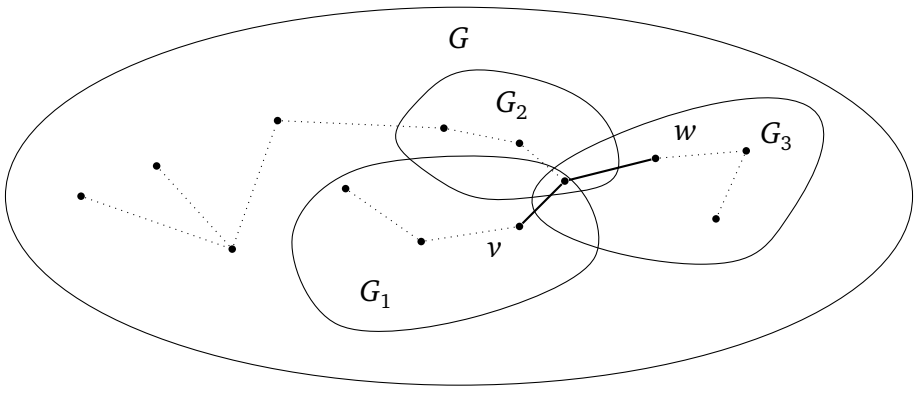

Figure 1: A valid path from v to w in bold. Note that G may contain nodes not belonging to any player.

3.2 Lower bound sketches and ideas

For item 1, we think this would require computing $\binom{k}{2}$ pairwise set disjointnes in the worst case, to determine where the intersections of each G_i are, at which point each player could on their own determine the possible paths between each of the subgraphs that intersect with their own.

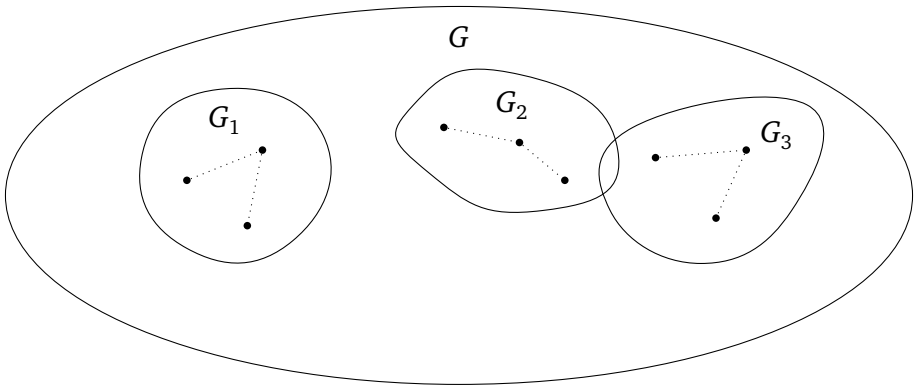

Figure 2: Worst case for item 1: Disjoint Subsets

For item 2, we would be required to compute $\binom{k}{2}$ pairwise intersections, since in order to minimize the path we would need to know all possible paths that could connect v and w. To find the shortest path, each player works backwards and sends the distances through their internal vertices to the players they intersect with, and each player will attempt to identify if a longer path can be dropped whenever they receive this list.

3.3 Lower bound using Equality

Equality problem. In the two-party Equality problem, Alice is given a string $x \in \{0,1\}^n$ and Bob is given a string $y \in \{0,1\}^n$. The goal is to determine whether $x = y$. It is known that Equality has deterministic communication complexity $\Omega(n)$.

Equality gadget. We construct a gadget that enforces equality at a single bit position. The gadget is shown in Figure 3. The gadget consists of two parallel branches.

1. The top branch corresponds to the assignment $x_i = y_i = 1$,
2. The bottom branch corresponds to the assignment $x_i = y_i = 0$.

Alice enables exactly one entry node of the gadget depending on x_i, and Bob enables exactly one exit node depending on y_i. A path exists through the gadget if and only if Alice and Bob select nodes on the same branch, which holds exactly when $x_i = y_i$.

Reduction to our graph problem. We construct a graph by chaining one copy of this gadget for each index $i \in [n]$ between a global start vertex v and end vertex w. In the

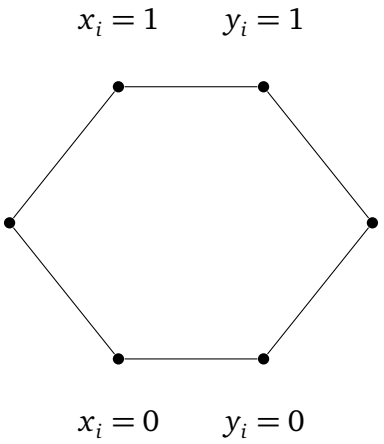

Figure 3: Gadget for proving equality. A path only exists if $x_i = y_i$ for either branch.

resulting graph, there exists a path from v to w if and only if all gadgets are traversable, which occurs if and only if $x_i = y_i$ for all i, i.e., if and only if $x = y$.

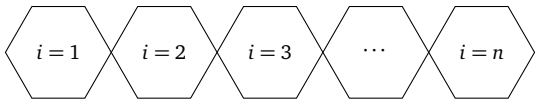

Figure 4: Equality gadgets chained as adjacent hexagonal modules.

Therefore, any protocol that solves the reachability problem on this graph can be used to solve the Equality problem. Since Equality requires $\Omega(n)$ deterministic communication, it follows that our graph problem also requires $\Omega(n)$ deterministic communication.

3.4 Lower bound using Disjointness

Set Disjointness problem. Alice is given a vector $x \in \{0,1\}^n$ and Bob is given a vector $y \in \{0,1\}^n$. The goal is to determine whether the sets represented by x and y are disjoint. It is known that Disjointness has deterministic communication complexity $\Omega(n)$.

Disjointness gadget. We construct a gadget that enforces disjointness at a single bit position. The gadget is shown in Figure 5. The gadget is constructed so that traversal is possible for all assignments except when both parties contain the element.

1. The top branch corresponds to when x_i has the element but y_i does not
2. The middle branch corresponds to when x_i and y_i do not have the element
3. The top branch corresponds to when y_i has the element but x_i does not

Reduction to our graph problem. We construct a graph by chaining one copy of this gadget for each index $i \in [n]$ between a global start vertex v and end vertex w. In the resulting

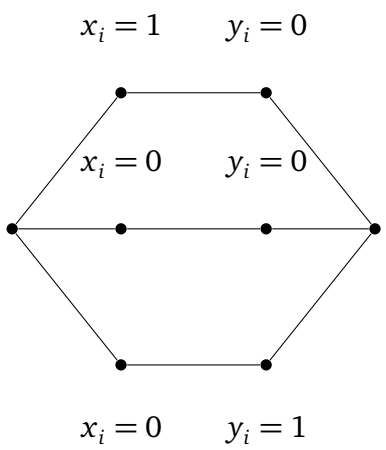

Figure 5: Gadget for proving equality, a path only exists if x_i and y_i do not have the same element

graph, there exists a path from v to w if and only if all gadgets are traversable, which occurs if and only if x and y does not contain the same element

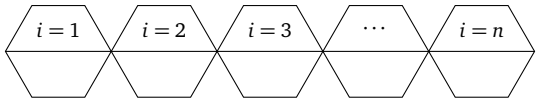

Figure 6: Equality gadgets chained as adjacent hexagonal modules.

Therefore, any protocol that solves the reachability problem on this graph can be used to solve the Disjoint problem. Since Disjoint requires $\Omega(n)$ deterministic communication, it follows that our graph problem also requires $\Omega(n)$ deterministic communication.

References